



中山大學
SUN YAT-SEN UNIVERSITY

操作系统实验报告

实验六：硬盘驱动与文件系统

姓 名： 郭盈盈
学 号： 24312063
教学班号： 吴岸聪老师班
专 业： 保密管理
院 系： 计算机学院

2025 学年第二学期

一. 实验目的

1. 理解块设备、分区表和文件系统之间的分层关系。
2. 掌握 MBR 分区表的格式，能够解析分区状态、类型、CHS 地址、起始 LBA 和扇区数量。
3. 理解 ATA PIO 工作方式，实现 28 位 LBA 下的磁盘识别与扇区读写。
4. 理解 FAT16 的 BPB、FAT 表、目录项、簇链和文件数据区，实现只读文件系统。
5. 将磁盘和文件系统接入内核，通过系统调用为 Shell 提供 `ls`、`open`、`read`、`close` 和 `cat` 功能。
6. 通过 `procfs`、`devfs`、`tmpfs` 等特殊文件系统理解 Linux“一切皆文件”的设计。

二. 实验环境

项目	配置
操作系统	YatSenOS / YYOS
开发语言	Rust (<code>no_std</code> 、 <code>no_main</code>)
目标架构	x86_64
引导环境	UEFI + OVMF
虚拟机	QEMU
磁盘接口	PATA，PIO 模式，28 位 LBA
文件系统	MBR + FAT16，只读实现
报告工具	Typst

三. 实验内容概述

本次实验从最底层的磁盘端口读写开始，逐层构造出可以被用户程序使用的文件访问接口。
整体调用关系如下：

```

Shell: ls / cat
|
用户态系统调用封装
|
内核 syscall dispatcher
|
Mount / FileSystem / FileHandle
|
FAT16: BPB、目录项、FAT 簇链
|
Partition: 分区内 LBA -> 磁盘 LBA
|
MBR 分区表
|
AtaDrive / AtaBus
|
ATA PIO 端口

```

实验中的存储 crate 定义了 `BlockDevice`、`PartitionTable`、`FileSystem`、`Read`、`FileHandle` 和 `Mount` 等抽象。内核中的 ATA 驱动只负责可靠地读写扇区，分区层负责地址平移，FAT16 层负责解释磁盘数据，系统调用层负责把这些能力安全地提供给用户程序。

四. 实验原理

1. MBR 分区表

MBR 位于磁盘第 0 个扇区，总长度为 512 字节。偏移 `0x1BE` 处开始存放 4 个主分区表项，每项 16 字节。一个表项包含：

偏移	长度	含义
<code>0x00</code>	1	状态， <code>0x80</code> 表示可启动
<code>0x01</code>	3	起始 CHS 地址
<code>0x04</code>	1	分区类型
<code>0x05</code>	3	结束 CHS 地址
<code>0x08</code>	4	分区起始 LBA，小端序
<code>0x0C</code>	4	分区扇区数量，小端序

CHS 地址中的扇区号占 6 位，柱面号占 10 位，因此不能直接按字节读取。以起始地址为例：

```
pub fn begin_sector(&self) -> u8 {
    self.data[0x02] & 0x3f
}

pub fn begin_cylinder(&self) -> u16 {
    ((self.data[0x02] as u16 & 0xc0) << 2)
    | self.data[0x03] as u16
}
```

后续实际磁盘访问采用 LBA，不依赖 CHS。分区是否存在也不能只通过 active 位判断，因为普通数据分区可以不是可启动分区。因此实现中使用“分区类型非零且扇区数量非零”判断有效表项。

2. 分区块设备

`Partition<T, B>` 本身也实现 `BlockDevice<B>`。对分区内第 `offset` 块的访问，需要转换为底层磁盘的：

$$\text{inner_offset} = \text{partition_start} + \text{offset}$$

同时必须检查 `offset < size`，并使用 `checked_add` 防止整数溢出。这样 FAT16 层只看到从 0 开始的逻辑分区，无须关心它位于整块磁盘的什么位置。

3. ATA PIO 与 28 位 LBA

ATA PIO 通过 I/O 端口与设备交互。主总线的常见基地址为 `0x1F0`，数据寄存器宽度为 16 位。发送一个 28 位 LBA 命令时：

1. 将扇区数量写入 `sector_count`。
2. 将 LBA 的低 24 位分别写入 `lba_low`、`lba_mid`、`lba_high`。
3. 将 LBA 的第 24 至 27 位写入 `drive` 寄存器低四位。
4. 在 `drive` 寄存器中设置 LBA 模式和主从盘选择。
5. 写入命令寄存器。
6. 轮询等待 `BUSY` 清零，并等待 `DATA_REQUEST_READY` 置位。

实现中的 `drive` 寄存器值为：

```
0xe0 | ((drive & 1) << 4) | (bytes[3] & 0x0f)
```

其中 `0xE0` 设置了固定高位和 LBA 模式，`drive` 选择主盘或从盘，最低四位保存 LBA 高四位。

ATA 每次通过 data 端口传输 16 位数据，因此读取一个 512 字节扇区需要读取 256 次 `u16`，并按小端序拆分为字节；写入过程与之相反。

4. FAT16 文件系统

FAT16 分区主要由保留区、FAT 表、根目录区和数据区构成：

```
[ BPB / Reserved ][ FAT 1 ][ FAT 2 ][ Root Directory ][ Data Clusters ]
```

BPB 位于分区第一个扇区，描述每扇区字节数、每簇扇区数、保留扇区数、FAT 数量、根目录项数量和每个 FAT 的扇区数等信息。

根目录占用扇区数为：

$$\text{root_dir_sectors} = \left\lceil \frac{\text{root_entries} \times 32}{\text{bytes_per_sector}} \right\rceil$$

第一个根目录扇区与第一个数据扇区分别为：

$$\text{root_start} = \text{reserved} + \text{fat_count} \times \text{sectors_per_fat}$$

$$\text{data_start} = \text{root_start} + \text{root_dir_sectors}$$

普通簇号从 2 开始，因此簇到扇区的转换公式为：

$$\text{sector} = (\text{cluster} - 2) \times \text{sectors_per_cluster} + \text{data_start}$$

FAT16 的每个 FAT 表项占 2 字节。通过当前簇号计算 FAT 表内字节偏移，可以读取下一个簇号。

`0xFFFF8..=0xFFFF` 表示簇链结束，`0xFFFF7` 表示坏簇，`0x0000` 表示空闲簇。

5. FAT16 目录项与文件读取

短文件名目录项大小为 32 字节，包含 8.3 文件名、属性、时间、首簇号和文件大小。本实验忽略长文件名项和卷标项，只解析标准短文件名。

读取目录时按目录占用的扇区遍历目录项：

- 首字节为 `0x00`：目录结束。
- 首字节为 `0xE5`：目录项已删除。
- 属性为 `LFN`：长文件名项，本实验跳过。
- 属性含 `VOLUME_ID`：卷标项，不作为普通文件返回。

读取文件时，需要同时考虑文件偏移、簇大小、扇区大小和用户缓冲区长度。每次复制的数据量为：

$$\min(\text{sector_remaining}, \text{request_remaining}, \text{file_remaining})$$

读完一个簇后查询 FAT 表进入下一个簇，直到缓冲区填满、文件结束或遇到簇链结束标记。

五. 实验过程与实现

1. 合并 storage 与 ATA 模块

我将实验六提供的增量代码合并到实验五工程中，并在 kernel 的 `Cargo.toml` 中加入 storage crate：

```
storage = { package = "yyos_storage", path = "../storage" }
```

同时在驱动模块中注册：

```
pub mod ata;  
pub mod filesystem;
```

2. 实现 MBR 分区表

`MbrTable::parse` 首先读取磁盘第 0 块，从 `0x1BE` 起每 16 字节解析一个分区项：

```
const PARTITION_TABLE_OFFSET: usize = 0x1be;  
const PARTITION_ENTRY_SIZE: usize = 16;  
  
for i in 0..4 {  
    let offset = PARTITION_TABLE_OFFSET + i * PARTITION_ENTRY_SIZE;  
    let entry = (&buffer[offset..offset + PARTITION_ENTRY_SIZE])  
        .try_into()  
        .map_err(|_| FsError::InvalidOperation)?;  
    partitions.push(MbrPartition::parse(entry));  
}
```

在 `MbrPartition` 中利用 `define_field!` 定义状态、磁头、分区类型、起始 LBA 和扇区数，并手工解析 CHS 中没有按字节对齐的扇区与柱面字段。

测试样例的解析结果为：

```

active      = true
begin head  = 1
begin sector = 1
begin cylinder = 0
partition type = 0x0b
end head    = 254
end sector  = 63
end cylinder = 764
begin LBA    = 63
total LBA    = 12289662

```

3. 实现 Partition 地址平移

分区块设备对越界访问返回 `FsError::InvalidOffset`，合法访问转换为底层磁盘 LBA：

```

let inner_offset = self
    .offset
    .checked_add(offset)
    .ok_or(FsError::InvalidOffset)?;
self.inner.read_block(inner_offset, block)

```

写操作采用相同的地址转换。虽然 FAT16 在本实验中只读，但完整实现分区写转发有助于保持 `BlockDevice` 抽象一致。

4. 实现 ATA 命令发送

`write_command` 将 28 位 LBA 写入寄存器，并轮询设备状态：

```

self.sector_count.write(1);
self.lba_low.write(bytes[0]);
self.lba_mid.write(bytes[1]);
self.lba_high.write(bytes[2]);
self.drive
    .write(0xe0 | ((drive & 1) << 4) | (bytes[3] & 0x0f));
self.command.write(cmd as u8);

```

发送命令后先判断空状态以识别不存在的设备，再等待 `BUSY` 清零，检查错误位，最后等待 DRQ 数据请求位。

5. 识别 ATA 磁盘

`identify_drive` 发送 `IdentifyDevice` 命令并读取 256 个 `u16`。根据 cylinder low/high 的签名区分 PATA、PATAPI、SATA 和 SATAPI。本实验只支持 PATA。

识别数据中的序列号、型号和最大 LBA 分别位于字节偏移 20、54 和 120。`AtaDrive` 只保存总线号、主从盘号、块数量、型号和序列号，实际端口由全局 `BUSES` 管理。

这种设计使 `AtaDrive` 可以轻量地实现 `Clone`，多个分区或文件系统对象可以持有同一个磁盘的逻辑句柄，而真实总线访问仍由 `Mutex<AtaBus>` 串行化。

6. 实现 PIO 扇区读写

读取数据时，每次从 data 端口读取一个 `u16`：

```
for chunk in buf.chunks_mut(2) {
    let bytes = self.read_data().to_le_bytes();
    chunk.copy_from_slice(&bytes[..chunk.len()]);
}
```

写入时将每两个字节组合成小端 `u16`。实现过程中曾出现 `bytes` 变量位于循环内部、`write_data` 位于循环外部的的问题，这会导致无法编译，也无法写出完整扇区。修正后每个 chunk 都在循环内写入 data 端口。

7. 实现 FAT16 BPB

使用 `define_field!` 完成 BPB 关键字段，包括：

- OEM 名称；
- 每扇区字节数、每簇扇区数；
- 保留扇区数、FAT 数量；
- 根目录项数量；
- 16 位与 32 位总扇区数；
- 每个 FAT 的扇区数；
- 隐藏扇区、卷标、系统标识；
- 末尾 `0xAA55` 签名。

`Fat16Bpb::new` 检查输入长度和尾部签名，避免把无效扇区解释为文件系统元数据。

8. 实现 DirEntry 与短文件名

`DirEntry::parse` 从 32 字节目录项中解析文件名、属性、创建/访问/修改时间、首簇和文件大小。`ShortFileName::parse` 将普通文件名转换为大写 8.3 格式，并检查空文件名、多个点、名称过长和非法字符。

例如 `hello.txt` 转换后：

```
basename = "HELLO "
extension = "TXT"
```

9. 实现 FAT 簇链和目录遍历

`next_cluster` 根据簇号定位 FAT 表项：

```
let offset = cluster.0 as usize * 2;
let sector = self.fat_start + offset / BLOCK_SIZE;
let index = offset % BLOCK_SIZE;
```

目录遍历会区分固定大小的 FAT16 根目录和由簇链组成的普通目录。路径查找使用 `/` 分割路径，逐级在当前目录中查找短文件名；若中间分量不是目录，则返回 `NotADirectory`。

10. 实现文件读取

`File` 保存当前偏移、当前簇、目录项和共享的 `Fat16Handle`。读取函数首先限制读取长度不超过文件剩余长度，然后根据当前偏移确定簇内扇区和扇区内偏移：

```
let cluster_offset = self.offset % cluster_size;
let sector_index = cluster_offset / BLOCK_SIZE;
let sector_offset = cluster_offset % BLOCK_SIZE;
```

读取扇区后，只复制当前请求需要的部分。恰好读完当前簇且文件尚未结束时，通过 FAT 表进入下一簇。这样既支持小于一个扇区的读取，也支持跨扇区、跨簇读取。

11. 将文件系统接入内核

内核初始化时打开第一个 ATA 磁盘，解析 MBR，取得第一个有效分区并挂载 FAT16：

```

let drive = AtaDrive::open(0, 0)
    .ok_or(FsError::DeviceError(DeviceError::UnknownDevice))?;
let table = MbrTable::parse(drive)?;
let part = table
    .partitions()?
    .into_iter()
    .next()
    .ok_or(FsError::InvalidOperation)?;

let fs = Fat16::new(part)?;
ROOTFS.call_once(|| Mount::new(Box::new(fs), "/".into()));

```

最初 `filesystem::init()` 没有加入内核启动流程，执行 `ls` 时 `ROOTFS.get().unwrap()` 会因 `None` 触发 `panic`。修复后在进程管理器初始化完成后调用 `filesystem::init()`，并把 `get_rootfs` 改为返回 `Option`。挂载失败只记录错误，`ls` 和 `open` 返回失败，不再终止内核。

12. 列出目录

添加 `ListDir` 系统调用。内核通过 `Mount::read_dir` 取得元数据迭代器，输出名称、大小和修改时间；目录名末尾添加 `/`：

NAME	SIZE	MODIFIED
EFI/	0.0 B	-
HELLO.TXT	38.0 B	2026-...

用户态 Shell 中：

```

fn cmd_list_dir(path: &str) {
    let path = if path.is_empty() { "/" } else { path };

    if !sys_list_dir(path) {
        println!("Failed to list directory '{}'.", path);
    }
}

```

13. 打开与读取文件

在进程资源表中增加 `Resource::File(FileHandle)`。 `open` 系统调用取得根文件系统中的文件句柄，并分配文件描述符； `read` 统一通过进程资源表分发； `close` 删除对应资源。

Shell 的 `cat` 遵循 `open-read-close`：

```

let fd = match sys_open(path) {
    Some(fd) => fd,
    None => {
        println!("cat: cannot open '{}'", path);
        return;
    }
};

loop {
    match sys_read(fd, &mut buf) {
        Some(0) => break,
        Some(count) => {
            sys_write(1, &buf[..count]);
        }
        None => break,
    }
}

sys_close(fd);

```

六. 实验结果

1. 单元测试

执行：

```
cargo test --package yyos_storage
```

结果：

```

running 4 tests
test fs::fat16::bpb::tests::test_fat16_bpb_1 ... ok
test fs::fat16::bpb::tests::test_fat16_bpb_2 ... ok
test fs::fat16::direntry::tests::test_dir_entry ... ok
test partition::mbr::entry::tests::partition_test ... ok

test result: ok. 4 passed; 0 failed

```

这说明 MBR 表项、两组 BPB 数据和 FAT16 目录项解析均通过测试。

2. 内核构建检查

执行：

```
cargo check --package yyos_kernel --lib
```

kernel 库目标通过类型检查。工程中仍有部分此前实验留下的未使用导入和未完成可选接口警告，但不影响实验六存储链路。

3. Shell 功能验证

在 FAT16 分区根目录放置 `hello.txt`，内容为：

```
Hello filesystem from 24312063!
```

启动系统后执行：

```
yyos> ls
NAME                SIZE MODIFIED
HELLO.TXT            32.0 B ...

yyos> cat /hello.txt
Hello filesystem from 24312063!
```

`ls` 能列出文件元信息，`cat` 能通过 `open-read-close` 读取并输出文件内容，说明 ATA 驱动、MBR、分区地址平移、FAT16 簇链和系统调用已经连通。

七. 探索 Linux 文件系统

本节按照实验任务直接回答 `procfs`、`devfs`、`tmpfs` 和 `chroot` 相关问题。

1. `procfs`

1.1. `/proc` 下的数字目录

`/proc/<pid>` 中的数字表示进程 ID。每个数字目录是内核为对应进程动态生成的视图，包含：

- `cmdline`：启动命令行；
- `environ`：环境变量；
- `status`、`stat`：进程状态与统计信息；
- `maps`、`smaps`：虚拟内存映射；
- `fd/`：文件描述符符号链接；

- `cwd`、`root`、`exe`：当前目录、根目录和可执行文件；
- `task/`：该进程的线程。

这些内容多数并不存储在磁盘上，而是读取时由内核根据当前状态生成。

1.2. `/proc/cpuinfo` 与 `/proc/meminfo`

`/proc/cpuinfo` 描述逻辑处理器，包括处理器编号、厂商、型号、主频、缓存、核心与线程拓扑、支持的指令集特性等。

`/proc/meminfo` 描述系统内存，包括总内存、空闲内存、可用内存、缓存、页缓存、交换空间、脏页、匿名页、Slab 和大页等。`free` 等工具会读取这些信息。

1.3. `/proc/loadavg` 与 `/proc/uptime`

`/proc/loadavg` 通常包含：

1. 最近 1、5、15 分钟的平均系统负载；
2. 当前可运行任务数/任务总数；
3. 最近创建进程的 PID。

负载并不等同于 CPU 利用率，它主要反映可运行或不可中断睡眠任务的数量。

`/proc/uptime` 包含两个以秒为单位的数值：系统启动后的运行时间，以及所有 CPU 空闲时间的累计值。

1.4. `/proc/interrupts`

该文件按中断号列出每个 CPU 处理该中断的次数、中断控制器类型和设备名称。可以观察时钟、键盘、网卡、磁盘等设备的中断分布，也可用于排查 IRQ 是否触发、是否集中在某个 CPU 等问题。

1.5. `/proc/self/status`

`self` 是指向当前读取进程自身 `/proc/<pid>` 的符号链接。`status` 以可读文本给出进程名、状态、PID/PPID、用户和组 ID、线程数、能力集合、CPU/内存亲和性、上下文切换次数以及 `VmSize`、`VmRSS` 等内存统计。

1.6. `/proc/self/smaps`

`smaps` 在 `maps` 的基础上对每个虚拟内存区域给出更细的统计，包括区域权限、映射文件、RSS、PSS、共享/私有的干净页和脏页、匿名页、交换空间、页大小等。它适合分析进程内存实际占用和共享情况，但读取开销比 `maps` 大。

1.7. `echo 1 > /proc/sys/net/ipv4/ip_forward`

该命令向 `procfs` 中的内核参数文件写入 `1`，启用 IPv4 转发，使 Linux 可以在不同网络接口之间转发 IPv4 数据包，常用于路由器、网关、NAT 和容器网络。

从系统调用角度看，Shell 依然使用普通的 `open`、`write`、`close`。VFS 将写操作转发给 `procfs` 对应节点，节点再修改内核网络参数。“一切皆文件”让应用可用统一接口操作配置、设备和内核状态，便于 Shell 组合、权限控制、重定向和脚本自动化，而不必为每个内核功能设计一套专用系统调用。

2. `devfs` / `devtmpfs`

2.1. 特殊设备文件

- `/dev/null`：读取立即得到 EOF，写入的数据被丢弃，常用于忽略输出。
- `/dev/zero`：读取返回无限的零字节，常用于初始化文件或匿名零页测试。
- `/dev/random`：提供内核随机数；现代 Linux 中初始化后通常与 `/dev/urandom` 具有相近的安全语义，但传统上可能在熵不足时阻塞。
- `/dev/urandom`：提供伪随机字节流，通常不阻塞，适合绝大多数密码学和一般随机数需求。

2.2. `/dev/kmsg`

`/dev/kmsg` 是用户空间访问内核日志缓冲区的字符设备。读取它可以看到内核输出的带序号、时间戳和日志级别的消息；`dmesg` 会通过相关接口读取内核日志。访问通常需要 `root` 或相应 `capability`，因为日志可能包含敏感的系统信息。

2.3. `/dev/sdX` 与 `/dev/sdX1`

`/dev/sdX` 表示整块 SCSI 风格块设备，例如 SATA、USB 存储或虚拟 SCSI 磁盘；`/dev/sdX1` 表示该磁盘上的第 1 个分区。前者包含分区表和全部扇区，后者只映射某个分区的 LBA 范围。

NVMe 系统中类似设备为 `/dev/nvme0n1`，其分区为 `/dev/nvme0n1p1`；virtio 磁盘可能是 `/dev/vda` 与 `/dev/vda1`。

2.4. `/dev/ttyX`、`/dev/loopX`、`/dev/srX`

- `/dev/ttyX`：虚拟控制台终端，例如 `tty1` 对应一个文本控制台。
- `/dev/loopX`：回环块设备，可把普通文件映射为块设备，用于挂载镜像文件。
- `/dev/srX`：SCSI CD/DVD 光驱设备，虚拟机挂载的 ISO 常表现为 `/dev/sr0`。

2.5. /dev/disk 中的符号链接

/dev/disk 常包含：

- `by-id`：按设备厂商、型号、序列号标识；
- `by-uuid`：按文件系统 UUID 标识；
- `by-label`：按文件系统卷标标识；
- `by-partuuid`：按分区 UUID 标识；
- `by-path`：按硬件连接路径标识。

这些符号链接最终指向 `/dev/sdX1`、`/dev/nvme...` 等设备。内核枚举顺序可能变化，`sda` 在下次启动时未必仍是同一磁盘；UUID、序列号或硬件路径更稳定，因此适合写入 `/etc/fstab` 和自动化脚本。

2.6. lsblk 输出解释

`lsblk` 以树状结构展示块设备及其父子关系。常见列包括：

- `NAME`：设备名；
- `MAJ:MIN`：主、次设备号；
- `RM`：是否可移除；
- `SIZE`：容量；
- `RO`：是否只读；
- `TYPE`：`disk`、`part`、`loop`、`rom`、`lvm` 等类型；
- `FSTYPE`：文件系统类型；
- `UUID`：文件系统 UUID；
- `MOUNTPOINTS`：挂载点。

磁盘节点下缩进的分区节点表示从属关系；如果分区上还有 LVM、加密或 RAID 映射，还会继续形成更深的设备树。

3. tmpfs

3.1. .pid 文件

PID 文件记录服务进程的 PID。程序启动时可原子地创建并锁定 PID 文件；若文件已被其他活动进程锁定，则拒绝再次启动。仅检查文件是否存在并不可靠，因为程序异常退出可能留下陈旧 PID 文件，因此通常还要配合文件锁或检查 PID 对应进程。

3.2. .lock 文件

锁文件用于协调多个进程对共享资源的访问。程序可以通过 `flock`、`fcntl` 文件锁或“排他创建文件”获得锁；只有持锁者可以操作资源，退出或关闭文件描述符后锁被释放。它常用于包管理器、串口设备、数据库和定时任务。

3.3. `.sock` / `.socket` 文件

这类文件通常是 Unix Domain Socket 的路径名。服务进程 `bind` 到该路径并监听，客户端通过同一路径连接，从而在本机进程间进行双向通信。相比 TCP，它不需要 IP 和端口，并可利用文件权限控制访问。

3.4. tmpfs 的作用

性能方面：数据主要保存在内存中，没有普通磁盘寻道和持久化写入开销，适合临时文件、共享内存、运行时状态和 socket。

安全方面：重启或卸载后内容消失，可减少临时敏感数据长期残留在磁盘上的风险；同时仍可使用 Unix 权限、挂载参数和命名空间进行隔离。但数据可能被交换到 swap，因此高安全场景仍需考虑加密和内存管理。

稳定性方面：运行时状态不污染持久磁盘，启动时总能得到干净环境；tmpfs 还可以设置容量上限，避免无限增长。不过它会占用内存和 swap，若缺少限制，大量写入仍可能造成内存压力。

4. chroot 前挂载 `proc`、`sys` 与 `dev`

命令：

```
mount proc /mnt/proc -t proc -o nosuid,noexec,nodev
mount sys /mnt/sys -t sysfs -o nosuid,noexec,nodev,ro
mount udev /mnt/dev -t devtmpfs -o mode=0755,nosuid
```

第一条把 `procfs` 挂载到新根目录，为 `chroot` 中的程序提供进程和内核状态；`nosuid`、`noexec`、`nodev` 限制其中内容被当作特权程序、可执行文件或设备节点使用。

第二条挂载 `sysfs`，使新系统能够看到设备模型、驱动、总线、电源和内核对象；示例中使用 `ro` 只读挂载，减少安装环境误修改内核设备属性的风险。

第三条挂载 `devtmpfs`，为新根目录提供控制台、终端、磁盘、随机数、`null` 等设备节点。实际安装还常绑定挂载 `/dev/pts`、`/dev/shm`、`/run`。

`chroot` 只改变进程看到的根目录，并不会自动创建这些虚拟文件系统。如果不挂载：

- `ps`、`top`、`free` 等依赖 `/proc` 的工具无法正常工作；
- `udev`、硬件探测和部分服务无法通过 `/sys` 获取设备信息；
- 缺少 `/dev/null`、`/dev/tty`、磁盘和伪终端，Shell、安装器和包管理器可能失败；

- DNS、终端会话、服务管理和设备配置能力会不完整。

八. 思考题

1. 1. storage crate 为什么使用条件 `no_std`？为什么 kernel 难以单元测试？

`#![cfg_attr(not(test), no_std)]` 表示正常编译 storage crate 时不链接标准库，只依赖 `core` 和 `alloc`，因此它可以运行在没有操作系统运行时的内核环境中；执行单元测试时则不启用 `no_std`，由 Rust 测试框架和宿主机标准库提供线程、输出和测试入口。

storage 中的 MBR、BPB、目录项等纯数据解析逻辑不依赖硬件，适合在宿主机进行单元测试。kernel 则较难测试，原因包括：

1. `no_std`、自定义入口、panic handler 和链接脚本与普通测试程序不同。
2. 端口 I/O、页表、中断、特权指令只能在 Ring 0 或虚拟机中执行。
3. 内核依赖 bootloader 提供内存映射、UEFI 表和已加载应用。
4. 全局中断、调度器和硬件状态使测试之间难以完全隔离。
5. 出错时可能直接 panic、三重故障或重启，无法使用普通测试框架收集结果。

因此合理做法是把可纯函数化的部分放入独立 crate 单元测试，把硬件和内核集成部分放入 QEMU 集成测试。

2. 2. MbrTable 泛型、Clone、PhantomData 与 Sized 的作用

`T: BlockDevice<B> + Clone` 表示 MBR 可以建立在任意块设备之上，例如真实 ATA 磁盘、内存磁盘或测试设备。解析分区后，每个 `Partition` 都需要持有同一个底层设备的逻辑句柄，因此需要克隆 `T`。

`B` 只出现在 trait 约束和方法签名中，没有直接存储在 `MbrTable` 字段里。`PhantomData<B>` 告诉编译器该结构在类型和生命周期语义上与 `B` 相关，也避免“未使用类型参数”错误。

`PartitionTable::parse` 返回 `Self`。trait 对象 `dyn PartitionTable` 的具体大小在编译期未知，不能直接按值返回，因此要求 `Self: Sized`，表示该方法只适用于大小已知的具体实现。

3. 3. AtaDrive 如何实现 Clone？分离 AtaBus 与 AtaDrive 的好处

`AtaDrive` 的字段是 `u8`、`u32` 和 `Box<str>`。整数可直接复制，`Box<str>` 实现了深拷贝，因此结构体可以派生 `Clone`。克隆的只是磁盘描述和总线索引，不是复制硬件。

`AtaBus` 持有真实端口对象并放在全局 `Mutex` 中，`AtaDrive` 只保存 bus/drive 编号和设备信息。这样设计的好处是：

- 端口访问集中管理，避免多个对象同时操作同一 ATA 总线；
- `AtaDrive` 轻量、可克隆，适合交给 MBR 和多个分区对象；
- 总线和设备职责分离，便于支持同一总线上的主盘与从盘；
- 上层只依赖 `BlockDevice`，不需要知道端口寄存器细节；
- 更容易使用内存块设备替换 ATA 驱动进行测试。

4. 4. 泛型、impl Trait 与 dyn Trait 的异同

4.1. 函数参数

`fn f<T: Foo>(f: T)` 使用显式泛型。调用者的具体类型在编译期确定，编译器会单态化，可在函数中引用类型参数 `T`，也能添加更多约束。

`fn f(f: impl Foo)` 也是静态分发，本质上是匿名泛型参数。写法简洁，但函数签名中不能直接命名该类型。每次调用仍只能传入一个具体类型。

`fn f(f: &dyn Foo)` 使用 trait object 和动态分发。参数由数据指针和虚表指针组成，可以在运行时接受不同具体类型，减少单态化代码膨胀，但有一次虚表调用开销，并受到 object safety 限制。

4.2. 结构体字段

`struct S<T: Foo> { f: T }` 的字段大小和具体类型在编译期已知，通常无堆分配和虚表开销，但 `S<A>` 与 `S<B>` 是不同类型。

`struct S { f: Box<dyn Foo> }` 在堆上存放具体对象，结构体本身大小固定，可以在运行时装入任意实现 `Foo` 的类型。实验中的 `Box<dyn BlockDevice<Block512>>` 和 `Box<dyn FileSystem>` 就利用了这种类型擦除。

5. 5. 硬链接、软链接与 Windows 快捷方式

硬链接是目录项直接指向同一个 inode。多个硬链接地位相同，共享文件内容和 inode 元数据；删除一个名字不会删除其他名字，只有链接计数归零且没有打开引用时才释放文件。硬链接通常不能跨文件系统，也通常禁止对目录创建。

软链接是一个独立 inode，其内容是目标路径。它可以跨文件系统，也可以指向目录；目标被删除或路径变化后会成为悬空链接。

Windows 快捷方式通常是 `.lnk` 普通文件，由 Shell 解释，除目标路径外还可保存图标、启动参数和工作目录。它更像桌面层元数据，不是所有文件 API 都会自动跟随。Linux 软链接由文件系统和 VFS 直接解析，对绝大多数程序透明。Windows 也存在 NTFS symbolic link、junction 和 hard link，它们才更接近 Linux 链接。

6.6. 日志文件系统与 FAT 的区别

FAT 这类非日志文件系统直接更新 FAT 表、目录项和数据。一次操作可能需要修改多个位置，如果写到一半断电，可能出现已分配簇没有目录项、目录项指向未完成簇链、重复分配或空间泄漏。恢复时通常需要 `fsck` 扫描较大范围的数据结构，耗时与文件系统容量和文件数量相关。

日志文件系统在修改主要结构前，先把事务意图或元数据更新写入日志，并通过提交记录确定事务是否完成。崩溃恢复时只需重放已提交事务、撤销未提交事务，能较快恢复一致状态。

日志实现需要事务、顺序保证、日志空间回收、校验和屏障等机制，代码更复杂，并增加额外写入。NTFS 使用日志保护关键元数据；一些 Linux 文件系统还支持 ordered、writeback 或 data journaling 等不同策略。日志通常保证文件系统结构一致，但不一定保证应用的最新数据已经持久化，应用仍需在适当位置调用 `fsync`。

九. 可选思考

1. 错误边界如何划分？

- 驱动层：报告超时、设备不存在、状态寄存器错误、读写失败和不支持的命令。
- 分区层：报告无效 MBR、越界 LBA、整数溢出和无有效分区。
- 文件系统层：报告 BPB/目录项损坏、坏簇、簇链循环、文件不存在、路径或类型错误。

最适合做面向用户的容错兜底的是文件系统/VFS 边界，因为它掌握操作语义，可以把底层错误转换为 `open`、`read`、`ls` 的失败；但底层各层仍必须尽早验证自身不变量，不能把越界和损坏数据继续向上传递。内核 syscall 层应保证错误不会演变为 panic。

2. FAT16 写入需要保证的一致性不变量

1. 一个已分配簇不能同时属于两个文件的簇链。

2. 目录项文件大小、首簇号和实际簇链必须一致。
3. FAT 链必须终止，不能形成意外环路或指向非法簇。
4. FAT 的多个副本应保持一致。
5. 删除文件时，目录项状态与簇释放不能产生长期悬挂或重复分配。
6. 扩展文件时，应先准备新簇，再以可恢复顺序连接簇链和更新目录项。

3. 教学实验优先采用哪种写回策略？

教学实验中我会先实现立即写回，因为语义直观、状态较少，便于验证每一步磁盘修改。待基本写功能稳定后，再引入小型日志或写回缓存。延迟写回虽然性能更好，但同时要求脏页管理、刷新时机、写入顺序和崩溃恢复，会显著扩大调试范围。

十. 实验总结

本次实验第一次让内核直接解释真实磁盘上的持久化数据。实现过程展示了操作系统存储栈清晰的层次：ATA 驱动只处理端口与扇区，MBR 将磁盘切分为分区，Partition 完成地址平移，FAT16 将簇链和目录项解释为文件，VFS 风格接口再把文件能力提供给系统调用和 Shell。

调试中最典型的问题是 ROOTFS 没有在启动阶段初始化。ls 从用户态一路进入内核后，最终对空的 `spin::Once` 执行 `unwrap`，导致整个系统 panic。这说明内核边界不能依赖“初始化一定成功”的假设。修复后，磁盘识别、分区解析和 FAT16 挂载都返回 `Result`，而系统调用对未挂载状态返回错误，系统即使无法访问磁盘也不会因此崩溃。

另一个重要认识是，磁盘上的结构必须严格按照规范解释。MBR 的 active 位不代表分区是否存在；FAT16 根目录与普通目录的存储方式不同；文件读取需要同时处理扇区边界、簇边界和文件长度。任何一个偏移或端序错误都会在更高层表现为完全不同的问题。

通过 Linux 特殊文件系统的探索，可以看到文件接口并不只用于持久化普通文件。内核状态、设备、进程信息、IPC 端点和运行时数据都可以映射为统一的路径与读写操作。这种统一抽象降低了用户程序和工具之间的组合成本，也是本实验中 `BlockDevice`、`FileSystem`、`Mount` 和 `Resource` 分层设计的现实对应。

十一. 参考资料

1. YatSenOS v2 Tutorial，实验六：硬盘驱动与文件系统，

<https://ysos.gzti.me/labs/0x06/tasks/>

2. OSDev Wiki, ATA PIO Mode ,
https://wiki.osdev.org/ATA_PIO_Mode
3. OSDev Wiki, FAT ,
<https://wiki.osdev.org/FAT>
4. Linux kernel documentation, procfs ,
<https://docs.kernel.org/filesystems/proc.html>
5. Linux manual pages: `proc(5)` 、 `random(4)` 、 `null(4)` 、 `tmpfs(5)` 、 `chroot(2)` °